Graphs

1 - Graphs and Graph Models

1.1 - Why do we have Graphs ?

Trees can't represent data where each nodes has more than 1 parent

1.2 - Undirected Graph

Defitnition

A graph $G = (V, E)$ consists of V- a non empty set of vertices (nodes) and E - a set of edges

Each edges has either one or two vertices associated with it, called its ***endpoints***. An edge is said to *connect* its endpoints

- If we have 1 endpoints, that means we are pointing to ourselves
- If we have 2 endpoints, that means we are traditional graphs with 2 endpoints and 1 edges

 - The set of edges E and the set of vertices V may be infinite => We called them Infinite Graph
 - Else, for finite set, we called them Finite Graph

Simple Graphs

A graph in which each edge connects 2 different vertices and where no 2 edges connect the same pair of vertices

Note that in a simple graph, each edge is associated with only 1 ***unordered pair of vertices*** and no other edges is related to this pair

When there is an edge of a Simple Graph associated to {u, v}, we can say that {u, v} is an edge of the graph

Multigraphs

A graph that contains multiple edges connecting to the same pair of vertices

When there are m different edges connecting to the same unordered pair of vertices $\{u, v\}$ $\implies$ We can say that $\{u, v\}$ is an edge of multiplicity of m



Loops

Sometimes, we need a graph to have edges that connect a vertex to itself => They are called loops

We also have graphs that may include loops, and possibly multiple edges connecting the same pair of vertices or a vertex to itself => They are called ***Pseudographs***

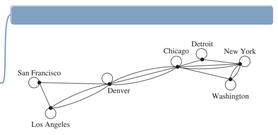


FIGURE 3 A computer network with diagnostic links.

1.3 - Directed Graph

In an Undirected Graph, the edges are referred to as ***Undirected***

In some cases, we need a Directed Graph

To model a Directed Graphs, which has edges made of ***"ordered pairs"*** (instead of "unordered pairs" in Undirected Graph)

Definition

A Directed Graph (or "digraph") (V, E) consists of a nonempty set of vertices V and a set of "Directed Edges" E. Each Directed Edge is associated with an **Ordered Pair** of vertices. The directed edges associated with the ordered pair (u, v) is said to "start" at u and "end" at v

Simple Directed Graphs

- A directed graphs has no loops and has no multiple directed edges
- It has at most 1 edge associated to each ordered pair (u, v)

 We call (u, v) an edge if there is an edge associated to it in the graph

Multiple Directed Graphs (Directed Multigraphs)

A graph that contains ***multiple directed edges*** from a vertex to a second (possibly the same) vertex

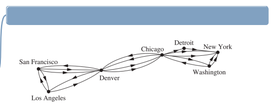


FIGURE 5 A computer network with multiple one-way links.

2 - Graph Terminology and Special Types of Graphs

2.1 - Basic Terminology

2 vertices u and v in an undirected graph G are called *adjacent*(or neighbors) in G if u and v are endpoints of an edge e of G. Such an edge e is called *incident*with the vertices u and v and e is said to *connectu* and v

The set of all neighbors of a vertex v of $G = (V, E)$ is called the *neighborhood*of v

The *degree of a vertex*in an *Undirected Graph*is the number of edges incident with it. A loop at a vertex contributes 2-unit of degrees of that vertex. The degree of vertex v is denoted as $deg(v)$

- A vertex of degree 0 is called ***Isolated***
- A vertex is called ***Pendant*** if and only if it has degree of 1

Theorem 1 - The Handshaking Theorem

Let $G = (V, E)$ be an undirected graph with m edges. Then

$$2m = \sum_{v \in V} deg(v)$$

(note that this applies even if multiple edges and loops are present)

Theorem 2

An undirected graph has an even number of vertices of odd degree

When (u, v) is an edge of the graph G with directed edges, u is said to be *adjacent to* v and v is said to be *adjacent from* u. The vertex u is called *initial vertex* of (u, v) and v is called the *terminal* or *end* vertex of (u, v). The initial vertex and terminal vertex of a loop are the same

Read the words carefully, as this is the case for Directed Graph only

In a directed graph, the *in-degree of a vertex* v, denoted by $deg^-(v)$, is the number of pointing edges to v. The *Out-degree of* v, denoted by $deg^+(v)$ is the number of edges pointing out from v

A loop at a vertex contributes 1-unit of both in-degree and out-degree of this vertex

Theorem 3

Let $G = (V, E)$ be a graph with directed edges. Then

$$\sum_{v \in V} deg^-(v) = \sum_{v \in V} deg^+(v) = |E|$$

Both of these sums are the number of edges in the graph

2.2 - Some Special Simple Graphs

Complete Graphs

Complete Graphs (a complete graphs on "n" vertices, denoted by K_n) is a simple graph that contains exactly 1 edge between each pair of nodes

A simple graph that consists of at least 1 pair of distinct nodes that is not connected is called ***Non-complete***

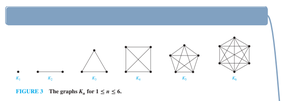


FIGURE 3 The graphs K_n for $1 \leq n \leq 6$.

EXAMPLE 6 **Cycles** A **cycle** C_n, $n \geq 3$, consists of n vertices $v_1, v_2, \ldots, v_n$ and edges $\{v_1, v_2\}, \{v_2, v_3\}, \ldots, \{v_{n-1}, v_n\}$, and $\{v_n, v_1\}$. The cycles C_3, C_4, C_5, and C_6 are displayed in Figure 4. ◀

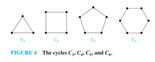


FIGURE 4 The cycles C_3, C_4, C_5, and C_6.

EXAMPLE 7 **Wheels** We obtain a **wheel** W_n when we add an additional vertex to a cycle C_n, for $n \geq 3$, and connect this new vertex to each of the n vertices in C_n, by new edges. The wheels W_3, W_4, W_5, and W_6 are displayed in Figure 5. ◀

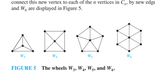


FIGURE 5 The wheels W_3, W_4, W_5, and W_6.

Bipartite Graphs

A simple graph G is called *bipartite* if its vertex set V can be partitioned into 2 disjoint sets V_1 and V_2 such that every edge in the graph connects a vertex in V_1 to a vertex in V_2 (so that no edge in G connects either 2 vertices in V_1 or 2 vertices in V_2). When this conditions are satisfied, we say that the pair (V_1, V_2) a *bipartition* of the vertex set V of G

Theorem 4

A simple graph is bipartite if and only if it is possible to assign one of 2 different colors to each vertex of the graph so that no 2 adjacent vertices are assigned of the same color

Complete Bipartite Graphs

A *Complete Bipartite Graphs* $K_{m,n}$ is a Bipartite Graph with 2 sets V_1 and V_2 where there exists n edges from 1 element in set V_1 connected to V_2 and there exists m edges from 1 element in set V_2 connected to V_1 (just like Neural Network)

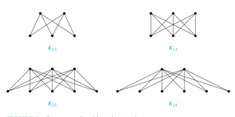


FIGURE 9 Some complete bipartite graphs.

Weighted Graphs

Essentially a type of graph with numerical values linked to all edges (even with negative values too but we won't working on that shit)

4 - Connectivity

4.1 - Paths and Cycles

Paths

For both Directed and Undirected Graphs, a path means a path with no duplication on both nodes and edges (ABSOLUTELY NO DUPLICATION)

Cycles

For both Directed and Undirected Graphs, a cycle means a path with no duplication on both nodes and edges (EXCEPT FOR THE INITIAL AND TERMINAL NODES)

Note: Since Undirected Graphs allows travel back and forth in the same node, for a cycle to be valid on an Undirected Graph, it must have 3 or more nodes

Note 2: This rule can be broken with Undirected Multigraph since 2 nodes can have 1 or more undirected edges

4.2 Connectedness in Undirected Graphs

An Undirected Graph is said to be ***Connected*** if there exists a path from any vertices to all vertices in the graph, otherwise, it is ***Disconnected***

4.3 - Connectedness in Directed Graphs

A Directed Graph is said to be ***Strongly Connected*** if there exists a path from any vertices to all vertices in the graph, otherwise, it is ***Weakly Connected*** (if we replace all directed edges to undirected edges in the graph)

5 - Graph Coloring

A coloring of a simple graph is the assignment of a color to each vertex of the graph so that no 2 adjacent vertices have the same color

The chromatic number of a graph is the least number of colors needed for the coloring of the graph. The chromatic number of a graph G is denoted by $\chi(G)$

The 4 color Theorem

The chromatic number of a planar graph is no greater than 4

7 - All Graph algorithms

Traversals

BFS - Breadth First Search

In BFS, we visit the first node first, then, we visit all adjacent nodes to this first node (for each adjacent node, we will visit all nodes that are adjacent to this node) => We continue to do this until all nodes are visited

Pseudocode - Table 10.4 (not Carrano book, chapter 10)

DFS - Depth First Search

bro idk how to explain this, you should see the diagram in chapter 10 (not Carrano book ofc) for better understand about this shit and read their pseudocode

If you hate recursion, please refer to the pseudocode in Carrano book, chapter 28, where they use iterative version

DO NOT USE CARRANO BOOK'S DIAGRAM, THEY ARE ABSOLUTELY SHIT

Finding shortest path

Dijkstra's Algorithm

Dijkstra's Algorithm is an algorithm used to find a shortest path from a specified node to all other nodes in the graph

How to understand this algorithm ? Please watch this video, I spent the whole fucking night to understand this mf with NotebookLM and Gemini, but they all gave me shits, so plz plz plz just watch the vid, it is peak

https://www.youtube.com/watch?v=bZkzHSx05KU

3 - Representing Graphs

3.1 - Adjacency Lists

Adjacency List is used to represent a graph with no multiple edges

It specifies vertices that are adjacent to each vertex of the graph

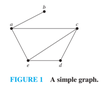


FIGURE 1 A simple graph.

TABLE 1 An Adjacency List for a Simple Graph.

Vertex	*Adjacent Vertices*
a	b, c, e
b	a
c	a, d, e
d	c, e
e	a, c, d



FIGURE 2 A directed graph.

TABLE 2 An Adjacency List for a Directed Graph.

Initial Vertex	*Terminal Vertices*
a	b, c, d, e
b	b, d
c	a, c, e
d	
e	b, c, d

3.2 - Adjacency Matrices

Suppose that $G = (V, E)$ is a simple graph where $|V| = n$. Suppose that the vertices of G are listed arbitrarily as $v_1, v_2, \ldots, v_n$. The **Adjacency Matrix A** (or A_G) of G is a $n \times n$ zero-one matrix with the following condition :

$$a_{ij} = \begin{cases} 1 \text{ if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 \text{ otherwise.} \end{cases}$$

EXAMPLE 3 Use an adjacency matrix to represent the graph shown in Figure 3.

Solution: We order the vertices as a, b, c, d. The matrix representing this graph is

$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

FIGURE 3 A simple graph.

EXAMPLE 4 Draw a graph with the adjacency matrix

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

with respect to the ordering of vertices a, b, c, d.

Solution: A graph with this adjacency matrix is shown in Figure 4.

FIGURE 4 A graph with the given adjacency matrix.

Note that an adjacency matrix of a graph is based on the ordering chosen for th Hence, there may be as many as $n!$ different adjacency matrices for a graph with n

The adjacency matrix for a simple graph is symmetric, that is $a_{ij} = a_{ji}$, because both entries are 1 when v_i and v_j are adjacent, and both are 0 otherwise. Moreover, this simple graph has no loop, therefore, entries $a_{ii} = 0$

Adjacency matrices can also be used to represents Undirected Graphs with loops and multiple edges, however, the matrix won't be a 0-1 matrix anymore because the entries are now representing the number of edges associated to $\{v_i, v_j\}$

EXAMPLE 5 **A pseudograph.**

Solution: The adjacency matrix using the ordering of vertices a, b, c, d is

$$\begin{bmatrix} 0 & 3 & 0 & 2 \\ 3 & 0 & 1 & 1 \\ 0 & 1 & 1 & 2 \\ 2 & 1 & 2 & 0 \end{bmatrix}$$

We used zero-one matrices in Chapter 9 to represent directed grap directed graph $G = (V, E)$ has a 1 in its (i, j)th position if there is an edge

The adjacency matrix for a Directed Graph $G = (V, E)$ is also the same for Undirected Graph, except that the pair of nodes is now ordered :

$$a_{ij} = \begin{cases} 1 \text{ if } (v_i, v_j) \text{ is an edge of } G, \\ 0 \text{ otherwise.} \end{cases}$$

- For Directed Graph, the adjacency matrix is not necessary to be symmetrical
- Same as Undirected Graph, the adjacency matrix can be able to represent Directed Multigraphs with loops

3.3 - Trade-off between Adjacency Lists and Adjacency Matrices

- For a simple graph with few edges, using Adjacency Matrix will create a ***Sparse Matrix*** => It is wiser to use Adjacency List than Adjacency Matrix
 - This is because if each vertex has a degree of not exceeding "c" ("c" << "n") => Each Adjacency List contains "c" or fewer vertices => The total items in all lists will be "c" * "n"
 - Comparing to n^2 items in adjacency matrix, this is an advantage for computation power
- For a "dense" simple graph with many edges (e.g: More than half of all possible edges) => Using adjacency matrix is better
 - For matrix, we can just use the index "i" and "j" and made 0-1 comparison to determine the existance of edge
 - But for Adjacency list, we have to search through the list to check => O(|V|) complexity

Depending on the task, each method has pros and cons

6 - Graph implementation in Java

COMP1020 prioritize using Adjacency List over Adjacency Matrix so that's what we will focus on

Some basic graphs operations (except some mfs like BFS, DFS, A*, Dijkstra)

Graph building operations

- add vertices
- add edges

Graph properties

- get number of vertices
- get number of edges
 - get total edges exist in this graph
- is directed
 - check if this graph is a directed graph
- is empty
 - check if this graph is empty or not

Edges and Neighbor Queries

- is edge
 - receive 2 nodes and check if there is an edge between these 2 nodes
- get edge
 - get an edge between 2 specific nodes
- edge iterator
 - return an iterator to get all edges related to this node

clear()

clear the graph

How to represent vertices and edges ?

2 classes

Edge<T>

Attributes

- endVertex
 - the terminal vertex of an edge
- weight
 - numerical value associated with an edge

Vertex<T>

Attributes

- value
- edgeList
 - storing the edges pointing at this vertex